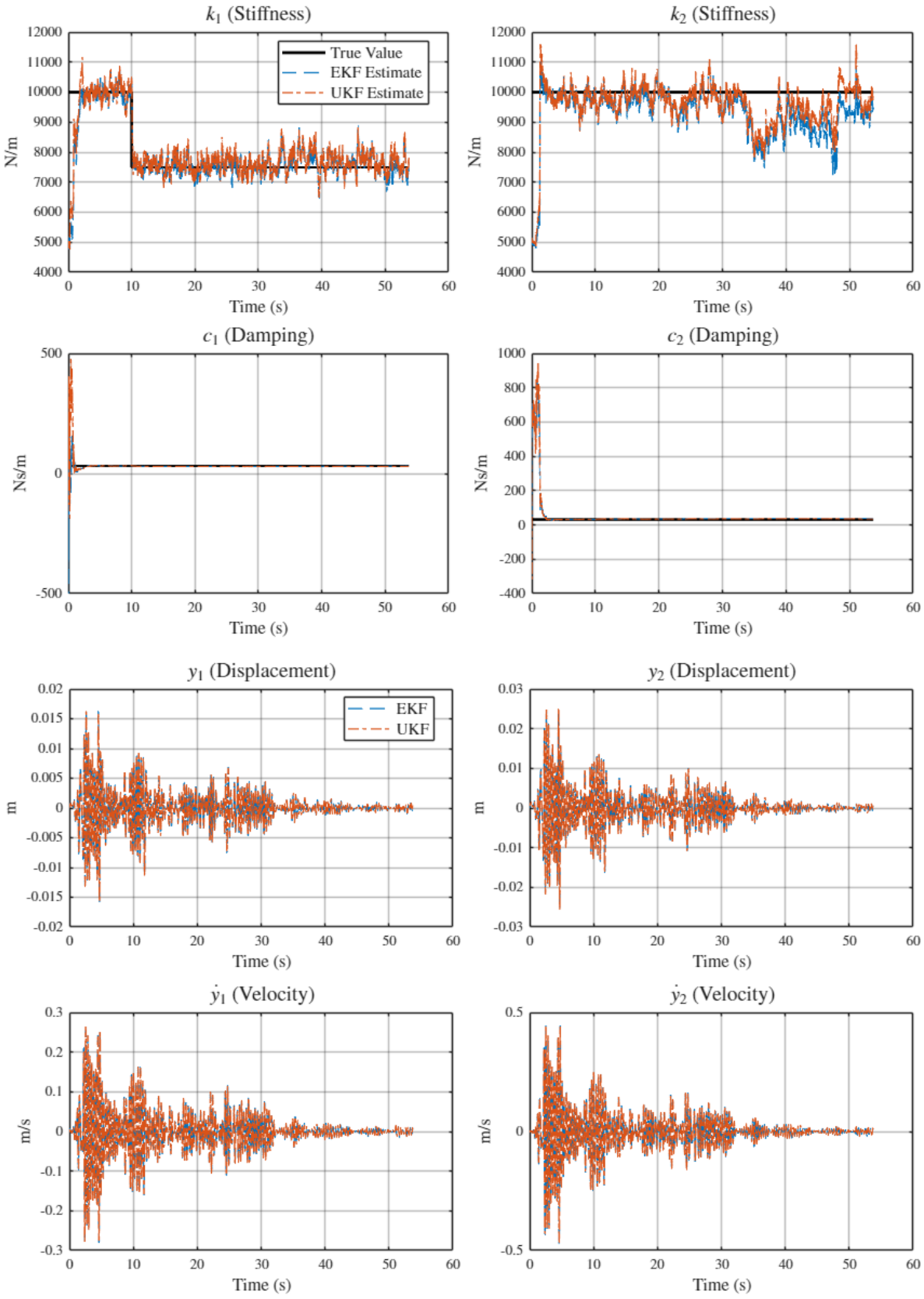
```
clearvars; clc; close all;

% 1. SYSTEM INITIALIZATION & PARAMETERS
% Structural masses
m1 = 10; m2 = 10;

% True system parameters for tracking
k1_initial = 10e3; k1_damaged = 7.5e3; t_fail = 10;
k2_true = 10e3;
c1_true = 31.76; c2_true = 31.76;

% Filter initial conditions (Based on Choice 2 for P0 and Choice 3 for Q)
X_init = [0; 0; 0; 0; 5000; 5000; 50; 50];
P_init = diag([1e-6, 1e-6, 1e-6, 1e-6, 1e6, 1e6, 1e6, 1e6]);
Q_noise = diag([0, 0, 0, 0, 5000, 5000, 5e-4, 5e-4]);
R_noise = diag([0.0438, 0.0967]);

% 2. DATA ACQUISITION
sensor_data = readmatrix('HW2_measurements.txt');
ground_motion = readmatrix('HW1_Q2_El_Centro.txt');

time_vec = sensor_data(:, 1);
accel_meas = sensor_data(:, 2:3)';
dt = mean(diff(time_vec));
num_samples = length(time_vec);

ug_accel = interp1(ground_motion(:,1), ground_motion(:,2), time_vec);

% 3. EXTENDED KALMAN FILTER (EKF) EXECUTION
est_X_EKF = zeros(8, num_samples);
est_P_EKF = zeros(8, 8, num_samples);
est_X_EKF(:, 1) = X_init;
est_P_EKF(:, :, 1) = P_init;

for i = 1:num_samples-1
    u_curr = ug_accel(i);
    u_mid = interp1(time_vec, ug_accel, time_vec(i) + 2*dt/3);

    % Predict Step via Ralston's Method
    [X_pred, F_mat] = predict_EKF_ralston(est_X_EKF(:, i), dt, u_curr,
u_mid, m1, m2);
    P_pred = F_mat * est_P_EKF(:, :, i) * F_mat' + Q_noise;

    % Update Step
    [Z_pred, H_mat] = get_measurement_and_jacobian(X_pred, m1, m2);
    Innovation_Cov = H_mat * P_pred * H_mat' + R_noise;
    Kalman_Gain = P_pred * H_mat' / Innovation_Cov;

    est_X_EKF(:, i+1) = X_pred + Kalman_Gain * (accel_meas(:, i+1) - Z_pred);
    est_P_EKF(:, :, i+1) = (eye(8) - Kalman_Gain * H_mat) * P_pred;
end
```

```
% 4. UNSCENTED KALMAN FILTER (UKF) EXECUTION
est_X_UKF = execute_UKF(time_vec, accel_meas, ug_accel, X_init, P_init,
Q_noise, R_noise, m1, m2, dt);

% 5. GENERATE TRUE PROFILES FOR PLOTTING
true_k1 = k1_initial * ones(num_samples, 1);
true_k1(time_vec >= t_fail) = k1_damaged;
true_k2 = k2_true * ones(num_samples, 1);
true_c1 = c1_true * ones(num_samples, 1);
true_c2 = c2_true * ones(num_samples, 1);

% 6. VISUALIZATION ENGINE
set(groot, 'defaultAxesTickLabelInterpreter','latex');
set(groot, 'defaultLegendInterpreter','latex');
set(groot, 'defaultTextInterpreter','latex');

c_true = '#000000'; c_ekf = '#0072BD'; c_ukf = '#D95319';

% FIGURE 1: Parameter Estimation
fig1 = figure('Name', 'Parameter Estimates', 'Position', [100, 100, 900,
600], 'Color', 'w');
t_layout1 = tiledlayout(2, 2, 'TileSpacing', 'compact', 'Padding',
'compact');

params = {true_k1, true_k2, true_c1, true_c2};
titles = {'$k_1$ (Stiffness)', '$k_2$ (Stiffness)', '$c_1$ (Damping)',
'$c_2$ (Damping)'};
ylabels = {'N/m', 'N/m', 'Ns/m', 'Ns/m'};
idxs = [5, 6, 7, 8];

for plt = 1:4
    nexttile;
    plot(time_vec, params{plt}, 'Color', c_true, 'LineWidth', 2.0); hold on;
    plot(time_vec, est_X_EKF(idxs(plt), :), '--', 'Color', c_ekf,
'LineWidth', 1.0);
    plot(time_vec, est_X_UKF(:, idxs(plt)), '-.', 'Color', c_ukf,
'LineWidth', 1.0);
    title(titles{plt}, 'FontSize', 14);
    ylabel(ylabels{plt}, 'FontSize', 12); xlabel('Time (s)', 'FontSize', 12);
    grid on; ax = gca; ax.GridAlpha = 0.3;
    if plt == 1
        legend('True Value', 'EKF Estimate', 'UKF Estimate', 'Location',
'best', 'FontSize', 11);
    end
end

% FIGURE 2: State Comparison
fig2 = figure('Name', 'State Estimates', 'Position', [150, 150, 900, 600],
'Color', 'w');
t_layout2 = tiledlayout(2, 2, 'TileSpacing', 'compact', 'Padding',
'compact');

titles_states = {'$y_1$ (Displacement)', '$y_2$ (Displacement)', '$
\dot{y}_1$ (Velocity)', '$\dot{y}_2$ (Velocity)'};
```

```
ylabels_states = {'m', 'm', 'm/s', 'm/s'};

for plt = 1:4
    nexttile;
    plot(time_vec, est_X_EKF(plt, :), '--', 'Color', c_ekf, 'LineWidth',
1.0); hold on;
    plot(time_vec, est_X_UKF(:, plt), '-.', 'Color', c_ukf, 'LineWidth',
1.0);
    title(titles_states{plt}, 'FontSize', 14);
    ylabel(ylabels_states{plt}, 'FontSize', 12); xlabel('Time (s)',
'FontSize', 12);
    grid on; ax = gca; ax.GridAlpha = 0.3;
    if plt == 1
        legend('EKF', 'UKF', 'Location', 'best', 'FontSize', 11);
    end
end

% LOCAL FUNCTIONS (Analytical Math & Filter Logic)

function [X_next, F_mat] = predict_EKF_ralston(X, dt, u_curr, u_mid, m1, m2)
    % Ralston's Integration with Analytical Jacobians
    f1 = calc_derivatives(X, u_curr, m1, m2);
    A1 = calc_analytical_jacobian(X, m1, m2);

    X2 = X + (2*dt/3) * f1;
    f2 = calc_derivatives(X2, u_mid, m1, m2);
    A2 = calc_analytical_jacobian(X2, m1, m2);

    X_next = X + dt * (0.25 * f1 + 0.75 * f2);

    % Discrete Transition Matrix
    I = eye(8);
    F_mat = I + dt * (0.25 * A1 + 0.75 * (A2 * (I + (2*dt/3) * A1)));
end

function dX = calc_derivatives(X, ug, m1, m2)
    % Continuous system dynamics
    dX = zeros(8, 1);
    dX(1) = X(3);
    dX(2) = X(4);
    dX(3) = (-(X(5)*X(1)) - X(6)*(X(1)-X(2)) - X(7)*X(3) - X(8)*(X(3)-
X(4))) / m1 - ug;
    dX(4) = (-X(6)*(X(2)-X(1)) - X(8)*(X(4)-X(3))) / m2 - ug;
end

function A = calc_analytical_jacobian(X, m1, m2)
    % Hardcoded Analytical Jacobian replacing syms
    A = zeros(8, 8);
    A(1, 3) = 1; A(2, 4) = 1;

    % Row 3 partial derivatives
    A(3, 1) = -(X(5) + X(6)) / m1;
    A(3, 2) = X(6) / m1;
    A(3, 3) = -(X(7) + X(8)) / m1;
```

```
    A(3, 4) = X(8) / m1;
    A(3, 5) = -X(1) / m1;
    A(3, 6) = -(X(1) - X(2)) / m1;
    A(3, 7) = -X(3) / m1;
    A(3, 8) = -(X(3) - X(4)) / m1;

    % Row 4 partial derivatives
    A(4, 1) = X(6) / m2;
    A(4, 2) = -X(6) / m2;
    A(4, 3) = X(8) / m2;
    A(4, 4) = -X(8) / m2;
    A(4, 6) = -(X(2) - X(1)) / m2;
    A(4, 8) = -(X(4) - X(3)) / m2;
end

function [Z, H] = get_measurement_and_jacobian(X, m1, m2)
    % Output mapping
    Z = [ (-(X(5)*X(1)) - X(6)*(X(1)-X(2)) - X(7)*X(3) - X(8)*(X(3)-X(4))) /
m1 ;
          (-X(6)*(X(2)-X(1)) - X(8)*(X(4)-X(3))) / m2 ];

    % Analytical Measurement Jacobian (Matches dynamics Rows 3 & 4)
    H = zeros(2, 8);
    H(1, :) = [-(X(5)+X(6))/m1, X(6)/m1, -(X(7)+X(8))/m1, X(8)/m1, -X(1)/m1,
-(X(1)-X(2))/m1, -X(3)/m1, -(X(3)-X(4))/m1];
    H(2, :) = [X(6)/m2, -X(6)/m2, X(8)/m2, -X(8)/m2, 0, -(X(2)-X(1))/m2, 0,
-(X(4)-X(3))/m2];
end

function ukf_res = execute_UKF(time_vec, z_meas, ug_accel, X0, P0, Q, R, m1,
m2, dt)
    % Unscented Kalman Filter Core
    N = 8; lambda = -7.97; alpha = 0.1; beta = 0;
    num_pts = 2 * N + 1;
    steps = length(time_vec);

    % Precompute weights
    w_mean = zeros(num_pts, 1); w_cov = zeros(num_pts, 1);
    w_mean(1) = lambda / (N + lambda);
    w_cov(1) = w_mean(1) + (1 - alpha^2 + beta);
    w_mean(2:end) = 1 / (2 * (N + lambda));
    w_cov(2:end) = w_mean(2:end);

    state = X0; covar = P0;
    ukf_res = zeros(steps, N);

    for k = 1:steps
        % Sigma Points via Cholesky
        L = chol((N + lambda) * covar, 'lower');
        sigmas = [state, state + L, state - L];

        % Propagate
        if k > 1
            for j = 1:num_pts
```

```
                f1 = calc_derivatives(sigmas(:, j), ug_accel(k-1), m1, m2);
                s2 = sigmas(:, j) + (2*dt/3) * f1;
                f2 = calc_derivatives(s2, ug_accel(k-1), m1, m2); % Midpoint
ug approx
                sigmas(:, j) = sigmas(:, j) + dt * (0.25 * f1 + 0.75 * f2);
            end
        end

        % Predict State
        x_pred = sigmas * w_mean;
        P_pred = Q;
        for j = 1:num_pts
            diff_x = sigmas(:, j) - x_pred;
            P_pred = P_pred + w_cov(j) * (diff_x * diff_x');
        end

        % Predict Measurement
        Z_sigmas = zeros(2, num_pts);
        for j = 1:num_pts
            [Z_sigmas(:, j), ~] = get_measurement_and_jacobian(sigmas(:, j),
m1, m2);
        end
        z_pred = Z_sigmas * w_mean;

        % Update
        S_cov = R; cross_cov = zeros(N, 2);
        for j = 1:num_pts
            diff_z = Z_sigmas(:, j) - z_pred;
            diff_x = sigmas(:, j) - x_pred;
            S_cov = S_cov + w_cov(j) * (diff_z * diff_z');
            cross_cov = cross_cov + w_cov(j) * (diff_x * diff_z');
        end

        K_gain = cross_cov / S_cov;
        state = x_pred + K_gain * (z_meas(:, k) - z_pred);
        covar = P_pred - K_gain * S_cov * K_gain';

        ukf_res(k, :) = state';
    end
end
```


k_1 (Stiffness)
True Value
EKF Estimate
UKF Estimate
N/m
Time (s)
k_2 (Stiffness)
N/m
Time (s)
c_1 (Damping)
Ns/m
Time (s)
c_2 (Damping)
Ns/m
Time (s)
y_1 (Displacement)
EKF
UKF
m
Time (s)
y_2 (Displacement)
m
Time (s)
$\dot{y}_1$ (Velocity)
m/s
Time (s)
$\dot{y}_2$ (Velocity)
m/s
Time (s)

Published with MATLAB® R2025b